mandate

Reproduction and Deployment

Source installation, local acceptance and public-network prerequisites

2026 FinTechathon Topic F: Agent Wallet

Mandate Labs

Nanyang Technological University, Singapore

Member	Role	Programme	Study year
Mengzhong Ma	Leader	Computational Finance	PhD, year 4
Sat Chidananda	Member	Computer Science	PhD, year 1
Vrushank Ajay Ahire	Member	Computer Science	PhD, year 1
Wilfried Ariel Mulyawan	Member	Computer Science	PhD, year 1

Mengzhong Ma	mengzhon001@e.ntu.edu.sg
Sat Chidananda	SAT001@e.ntu.edu.sg
Vrushank Ajay Ahire	vrushank001@e.ntu.edu.sg
Wilfried Ariel Mulyawan	wmulyawa001@e.ntu.edu.sg

Advisor: Bao Te, Nanyang Technological University.

Correspondence: Mengzhong Ma, team leader.

Member names, roles, programmes and study years reproduce the submitted registration record. The source registration files are not part of this public document.

Document revision: LaTeX revision 1, 20 October 2026.

Implementation reference: tested source commit

`29d10422954d36f0e37c2d89336986d10d014aba`

https://github.com/MandateLabs666/mandate-wallet

This is a testnet engineering document. Observed results, implementation assumptions and research extensions are identified separately. Preparing this document does not constitute an official competition submission.

Contents

1 Source identity and prerequisites 2

2 Build and run the local purchase case 2

3 Guided setup and graphical Claude Desktop 2

4 Test suites and expected scope 3

5 Existing public deployments and read-only verification 3

6 Fresh testnet deployment and recovery design 4

7 Operational acceptance and publication boundary 4

1 Source identity and prerequisites

Clone https://github.com/MandateLabs666/mandate-wallet. The tested source tag `fintechathon-2026-source` points to commit `29d10422954d36f0e37c2d89336986d10d014aba`. Later material-only publication changes have their own record. This guide describes the source-mode testnet release; SDK publication to npm and a signed independent installer are not assumed.[2, 3]

Use Node 24.x, at least 24.2, pnpm 11.25.0, Python 3.12+ and Foundry 1.5.1. The account build pins Solidity 0.8.26 with optimizer 200, viaIR and Cancun. Bootstrap downloads only pinned public dependency archives and validates their hashes. Modified dependency bytes cause a failure rather than a silent overwrite. The frozen lockfile controls package resolution.

The local demo does not need a synchronized execution/consensus node, a personal wallet or public funds. Public read-only checks use configured third-party RPCs. Public transaction workflows additionally require dedicated roles, test assets and explicit owner intent review. Ordinary reproduction must not start a Geth/Lighthouse synchronization job.

2 Build and run the local purchase case

In a new checkout, run:

```
python3 scripts/bootstrap-sdk.py
CI=true pnpm install --frozen-lockfile
pnpm sdk:build
pnpm sdk:demo artifacts/reproduction/demo
```

The build compiles pinned x402 workspace packages, the formal account, MCP gateway, verifier and Web assets, then checks TypeScript. The local example starts temporary Anvil and controlled HTTP components, uses mock USDC and public local-test roles, and tears down its own resources.

The summary must show original-hash recovery, passing required independent checks, confirmed fixture revocation and rejection of the later step. The verifier should retain overall=incomplete for missing HTTPS source and full-history guarantees. A successful local summary is not evidence that a new public network or third-party merchant combination passed.

3 Guided setup and graphical Claude Desktop

After building, start the source-mode setup component:

```
pnpm setup:build
pnpm setup:start
```

Open the printed loopback URL, normally http://127.0.0.1:18743/setup.html. The first step lets the user select Local practice, Testnet or an existing configuration. Local mode prepares an isolated test account and mock token. Testnet mode connects an external owner wallet and prepares separate deployment, gas funding, token funding, root policy and finite session transactions. Each requires the owner's review in the wallet.

The selected profile has independent local files, database, connection descriptor and Guided identity. In the agent step choose Claude Desktop, download that profile's Guided package, import it under Claude Settings / Extensions and enable it. A correctly bound Guided package has no required Agent profile or Connection file fields. Those fields belong to the retained advanced manual extension. Host installation policies and tool approval still apply.

Ask Claude to invoke only `mandate_get_context` first. Compare chain, wallet, root, session, current authority, budget and balance with the workspace before procurement. The wizard's independent local connectivity probe is useful but does not establish a model invocation inside Claude. After host verification, request one configured service and inspect payment, delivery and result separately.[4]

The manual route remains available for existing Gateway profiles. Never upload an owner private key as a connection file. A connector selects access to an existing grant; it does not create authority or make the connected browser wallet an account owner. Resetting guide progress preserves accounts and orders and cannot undo chain transactions. Switching profiles does not rewrite an installed MCP binding.

4 Test suites and expected scope

Run the local suites in independent output directories:

```
pnpm m3:test:contracts
pnpm m3:test
pnpm m4:test
pnpm m5:test
pnpm m6:test
pnpm m7:test
python3 scripts/bootstrap-echo-tests.py
pnpm m2:test:contracts
pnpm m2:test
pnpm setup:test
pnpm exec vitest run --config mandate/desktop/vitest.config.ts
```

Formal contracts cover root/session bounds, signatures, versioning, expiry and independent reservation invariants. Service tests exercise real local EVM/HTTP recovery. Evidence tests cover disclosure and retention/controller boundaries. Web tests cover owner/status transitions and presentation logic. Operations tests cover workers, wire faults, reorg and diagnostics. Onboarding and desktop tests cover profile/package/connection behavior. These tests do not replace the separately recorded native Rabby/Claude run or remaining human-device acceptance.

The accepted code snapshot passed 50 Solidity cases and 153 TypeScript tests in 16 reproduction phases. Two fuzz suites report 10001 runs each and the reservation invariant reports 8192 calls with no handler revert. These test counts are distinct from the research dataset's 10000 explicitly invalid eth_call samples. Shared host toolchains and verified public caches are declared. Personal runtime and parent workspace dependencies were not copied.[3]

5 Existing public deployments and read-only verification

Base Sepolia (84532) uses native test USDC. Ethereum Sepolia (11155111) has controlled-token fixture evidence, not an assertion that every token there has the required contract-signature route. Use `mandate/sdk/VERSIONS.json`, network manifests and each run's explicit account/code fingerprints. The recorded Claude case uses a later owner-created account in the same factory family, so the historic M3 account address is not the video account.[1]

Check chain ID, account code, constructor/immutable context, asset domain and configured role addresses. A copied bundle's own trust assertions do not independently establish those facts. A token proxy update can change behavior even when its address or domain is unchanged. General wallet-compatibility documentation is a design reference; direct settlement evidence is still required.[5]

The public-case verifier command is read only:

```
node --import tsx mandate/evidence/cli.ts \
  --bundle competition/video/claude-desktop-base-v1/execution-evidence/purchase-bundle.json \
  --trust competition/video/claude-desktop-base-v1/execution-evidence/verifier-trust.json \
  --output /tmp/mandate-public-verification.json
```

Inspect the trust file before running it. Expected required financial checks pass in the archived report, while overall remains incomplete. A provider that cannot supply historical state must leave verification unresolved rather than induce another payment.

6 Fresh testnet deployment and recovery design

Use a dedicated new workspace and private journal, preserve historical manifests, and prepare separate owner, session, registration-relayer and facilitator roles. Check current test balances and exact chain/token compatibility. The onboarding source workflow is the simplest owner-facing path; historical deployment harnesses retain their original prerequisites and should not be run over existing evidence as if they were a stateless service.

Persist each deployment/funding/authorization intent, sender nonce, raw signed operation and expected transaction hash before broadcast. After an RPC timeout reconcile that original operation before retrying. A deployed account whose receipt is temporarily unavailable is not a reason to deploy another account. Never copy another user's signing journal or reuse its nonce with a new identity.

Review token procurement limits separately from gas funding. A payment signature can leave the client only after its exact registration is confirmed. A five-minute payment authorization can expire inside a longer session. An expired unpaid business step cannot be reopened by changing the nonce; safe release and a genuinely new owner-approved purchase are separate actions.

Mainnet writes, paid external services, unsupported schemes and new merchant/facilitator combinations require separate acceptance and authorization. The selected service-payment route does not use EntryPoint for every purchase. Echo's strict bundler deployment is a separate reference workflow and is not a prerequisite for ordinary Mandate local setup.

7 Operational acceptance and publication boundary

Keep private keys, connection tokens, SQLite stores, raw signed transactions and bound packages in private runtime directories. Public receipts/evidence signatures are different from reusable payment-authorization payloads. Source packages exclude registration forms and private proposal originals. Retain upstream licenses and notices rather than replacing them with the project's MIT label.

Current gaps include signed standalone distribution and updates, persistent local-node recovery, remaining operating-system/device paths, independent paid-merchant source witnessing and external human audit. Publishing documentation and source does not close those tasks, authorize mainnet use or submit the competition materials. The authoritative task checklist preserves their status.

References

[1] Mandate Labs. Native Claude Desktop, Rabby and Base Sepolia execution evidence. https://github.com/MandateLabs666/mandate-wallet/tree/29d10422954d36f0e37c2d89336986d10

d014aba/competition/video/claude-desktop-base-v1/execution-evidence, 2026. Recorded 19 October 2026; public evidence, trust configuration and source fingerprints.

[2] Mandate Labs. Mandate Wallet: tested source snapshot. https://github.com/MandateLabs666/mandate-wallet/tree/29d10422954d36f0e37c2d89336986d10d014aba, 2026. Source commit 29d10422. Accessed 20 October 2026.

[3] Mandate Labs. Clean-source reproduction and publication records. https://github.com/MandateLabs666/mandate-wallet/tree/a0156351bb2b0eca183c58b3071a3aae52cdbb94/artifacts/competition/source-delivery/submission20261020, 2026. 16 phases, 50 Solidity cases and 153 TypeScript tests; source commit 29d10422.

[4] Model Context Protocol contributors. Model Context Protocol Specification. https://modelcontextprotocol.io/specification/2025-11-25, 2025. Protocol reference dated 2025-11-25; repository pins SDK 1.27.1. Accessed 20 October 2026.

[5] x402 contributors. Wallet Compatibility. https://docs.x402.org/advanced-concepts/wallet-compatibility, 2026. Accessed 20 October 2026; actual token and facilitator behavior must be tested.